EAST WEST UNIVERSITY

Sprint 2

Title: **Solar Man**: A Web-Based Solar Plant Installation and Management System

Course title: - Software Engineering

Course code: - CSE412

Section: 01

Semester: Fall 26

Group number: 06

Date of submission: 19-10-2026

Submitted To:

Dr Mohammad Mahdi Hassan

Associate Professor

Department of Computer Science and Engineering

East West University

Submitted by:

Name	Student ID
Shakil Islam	2023-3-60-022
Arnob Paul	2024-3-60-257
Rakib Al Hasan	2024-1-60-143
Anjuman Ara	2024-1-60-279

CSE412 — Sprint 2

Design Pattern Implementation

1. Introduction

Design patterns are reusable solutions to common software design problems. They help developers organize code, reduce duplication, and make applications easier to maintain and extend.

For the Solar Man project, two design patterns are selected:

1. **Builder Pattern** — for configuring solar systems.
2. **Strategy Pattern** — for handling different customer payment methods.

These patterns are demonstrated using Python and can be integrated into the project's web application.

2. Builder Pattern

2.1 Definition

The Builder Pattern is a creational design pattern that constructs a complex object step by step. It separates the construction process from the final object representation.

In Solar Man, customers may require different solar system configurations depending on their electricity consumption, budget, and backup requirements.

The Builder Pattern allows the application to assemble different configurations using a consistent construction process.

2.2 Application in Solar Man

The pattern contains the following components:

- **Product:** SolarSystem
- **Builder:** SolarSystemBuilder
- **Concrete Builders:** BasicSolarSystemBuilder and PremiumSolarSystemBuilder
- **Director:** SolarSystemDirector
- **Client:** The code that selects a builder and requests a configuration.

The Director controls the sequence of construction steps, while each concrete builder specifies the components of the solar system.

2.3 Python Implementation

```
from dataclasses import dataclass

@dataclass
class SolarSystem:
    panels: str = ""
    inverter: str = ""
    battery: str = ""
    mounting_structure: str = ""
    monitoring: bool = False

    def display(self):
        print("Solar System Configuration")
        print(f"Panels: {self.panels}")
        print(f"Inverter: {self.inverter}")
        print(f"Battery: {self.battery}")
        print(f"Mounting: {self.mounting_structure}")
        print(f"Monitoring: {self.monitoring}")

class SolarSystemBuilder:
    def __init__(self):
        self.system = SolarSystem()

    def build_panels(self):
        raise NotImplementedError

    def build_inverter(self):
        raise NotImplementedError

    def build_battery(self):
        raise NotImplementedError

    def build_mounting(self):
```

```
        raise NotImplementedError

    def get_solar_system(self):
        return self.system

class BasicSolarSystemBuilder(SolarSystemBuilder):
    def build_panels(self):
        self.system.panels = "Basic solar panels"

    def build_inverter(self):
        self.system.inverter = "Standard inverter"

    def build_battery(self):
        self.system.battery = "Optional battery"

    def build_mounting(self):
        self.system.mounting_structure = "Standard mounting"

class PremiumSolarSystemBuilder(SolarSystemBuilder):
    def build_panels(self):
        self.system.panels = "High-efficiency solar panels"

    def build_inverter(self):
        self.system.inverter = "Hybrid inverter"

    def build_battery(self):
        self.system.battery = "Backup battery"

    def build_mounting(self):
        self.system.mounting_structure = "Reinforced mounting"
        self.system.monitoring = True

class SolarSystemDirector:
    def construct_system(self, builder):
```

```
        builder.build_panels()
        builder.build_inverter()
        builder.build_battery()
        builder.build_mounting()
        return builder.get_solar_system()

# Client code
director = SolarSystemDirector()

basic_builder = BasicSolarSystemBuilder()
basic_system = director.construct_system(basic_builder)

print("BASIC SOLAR SYSTEM")
basic_system.display()

print()

premium_builder = PremiumSolarSystemBuilder()
premium_system = director.construct_system(premium_builder)

print("PREMIUM SOLAR SYSTEM")
premium_system.display()
```

2.4 Expected Output

BASIC SOLAR SYSTEM
Solar System Configuration
Panels: Basic solar panels
Inverter: Standard inverter
Battery: Optional battery
Mounting: Standard mounting
Monitoring: False

PREMIUM SOLAR SYSTEM
Solar System Configuration

Panels: High-efficiency solar panels

Inverter: Hybrid inverter

Battery: Backup battery

Mounting: Reinforced mounting

Monitoring: True

2.5 Role of the Director

The SolarSystemDirector defines the construction sequence. The client does not need to call each building method individually.

With a Director:

- Construction logic is centralized.
- The order of construction steps is consistent.
- Different builders can follow the same workflow.

Without a Director:

- The client must call every construction method.
- Construction steps may be duplicated or omitted.
- Different parts of the application may follow different sequences.

2.6 Advantages

1. Separates the construction process from the final object.
2. Supports multiple solar system configurations.
3. Reduces duplicated construction logic.
4. Improves maintainability and extensibility.

2.7 Limitations

The Builder Pattern introduces additional classes and may be unnecessary for very simple objects. Solar component compatibility, system sizing, and cost calculations must still be implemented separately.

3. Strategy Pattern

3.1 Definition

The Strategy Pattern is a behavioral design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable.

In Solar Man, customers may use different payment methods. Instead of placing all payment logic inside the checkout class, each method can implement a common payment interface.

3.2 Application in Solar Man

The pattern contains the following components:

- **Strategy:** PaymentStrategy
- **Concrete Strategies:** OnlinePayment and CashPayment
- **Context:** PaymentContext
- **Client:** The code that selects a payment method.

The payment context delegates the payment operation to the selected strategy.

3.3 Python Implementation

```
from abc import ABC, abstractmethod

class PaymentStrategy(ABC):

    @abstractmethod
    def pay(self, amount):
        pass

class OnlinePayment(PaymentStrategy):

    def pay(self, amount):
        # A real payment gateway must be integrated here.
        print(
            f"Online payment initiated for "
            f"BDT {amount:.2f}"
        )

class CashPayment(PaymentStrategy):
```

```
    def pay(self, amount):
        # Cash must be verified before confirming payment.
        print(
            f"Cash payment recorded as pending verification: "
            f"BDT {amount:.2f}"
        )
class PaymentContext:

    def __init__(self, strategy=None):
        self.strategy = strategy

    def set_strategy(self, strategy):
        self.strategy = strategy

    def checkout(self, amount):
        if self.strategy is None:
            raise ValueError(
                "Please select a payment method."
            )

        if amount <= 0:
            raise ValueError(
                "Payment amount must be positive."
            )

        self.strategy.pay(amount)

# Client code
payment = PaymentContext()

payment.set_strategy(OnlinePayment())
payment.checkout(25000)
print()
```

```
payment.set_strategy(CashPayment())
payment.checkout(10000)
```

3.4 Expected Output

Online payment initiated for BDT 25000.00

Cash payment recorded as pending verification: BDT 10000.00

3.5 Explanation

The PaymentStrategy abstract class defines a common interface for all payment methods. OnlinePayment and CashPayment implement the same interface but handle their respective workflows differently.

The PaymentContext selects and executes the strategy. As a result, the checkout code does not need separate payment-processing logic for every payment method.

For example, a future supported payment method could be added by creating another class that implements PaymentStrategy.

3.6 Advantages

1. Separates payment methods into independent classes.
2. Reduces conditional logic in the checkout workflow.
3. Makes it easier to add supported payment methods.
4. Improves testing and maintainability.
5. Supports the Open–Closed Principle by allowing extension through new strategies.

3.7 Limitations

Each payment method requires its own implementation. Real online payments also require secure gateway integration, transaction verification, error handling, and appropriate security controls.

Technician payouts should be managed through a separate payout and approval workflow rather than being treated as customer checkout payments.

4. Comparison of the Two Patterns

Feature	Builder Pattern	Strategy Pattern
Pattern category	Creational	Behavioral
Main purpose	Constructs complex objects	Interchanges algorithms or behaviors
Solar Man application	Solar system configuration	Customer payment methods
Main class	SolarSystemDirector	PaymentContext
Main benefit	Consistent, reusable construction	Flexible, interchangeable payment behavior

5. Conclusion

The Builder Pattern and Strategy Pattern provide useful solutions to two design problems in Solar Man.

The Builder Pattern separates solar system construction from the specific configuration being created. It allows the application to assemble different solar system options through a consistent process.

The Strategy Pattern separates customer payment methods into independent implementations. It makes the checkout workflow easier to extend and maintain.

Together, these patterns demonstrate how object-oriented design principles can improve the structure, flexibility, and maintainability of a Python-based web application.